

开源软件基础

编译器测试

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

November 26, 2025



Outline

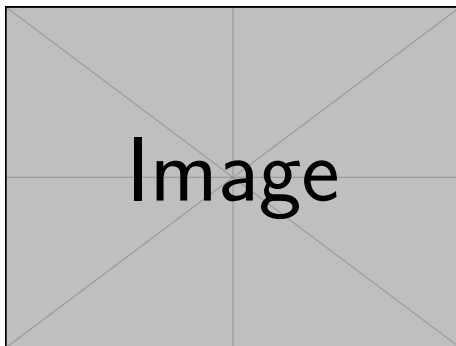
- 1 背景
- 2 编译器测试概述
- 3 差分测试
- 4 CSmith 和 Yarpgen

Outline

- 1 背景
- 2 编译器测试概述
- 3 差分测试
- 4 CSmith 和 Yarpgen

编译器测试概述

- 编译器正确性的重要性
- 传统测试方法的局限性
- 随机测试的优势
 - 覆盖更多边界情况
 - 发现隐藏缺陷
 - 自动化程度高



看起来高端但其实



Figure 2: 知乎经典问题

Outline

- 1 背景
- 2 编译器测试概述
- 3 差分测试
- 4 CSmith 和 Yarpgen

差分测试

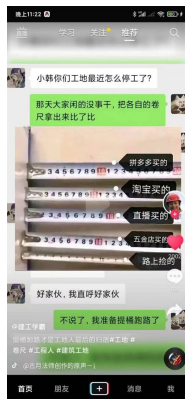


Figure 3: 提桶跑路

面向编译器的差分测试

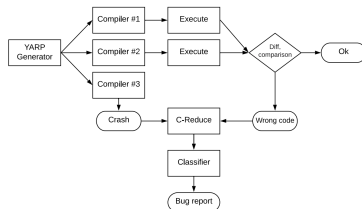


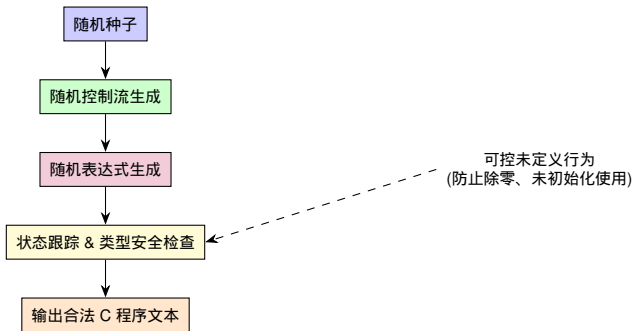
Figure 4: Yarpngen 工作流程

Outline

- 1 背景
- 2 编译器测试概述
- 3 差分测试
- 4 CSmith 和 Yarpgen

Csmith 原理示意 (随机生成 + 类型安全)

- **Csmith**: 自动生成合法 C 程序, 用于编译器测试
- 核心原理:
 - ① 随机控制流与表达式生成 (if, for, while, switch, 运算符)
 - ② 状态跟踪 & 类型安全保证 (变量初始化、数组/指针访问检查)
 - ③ 输出 C 代码文本, 可通过随机种子重复生成



BuggyReturnPass

```
// BuggyReturnPass: replace all integer returns with constant 1
struct BuggyReturnPass : public PassInfoMixin<BuggyReturnPass> {
    PreservedAnalyses run(Module &M, ModuleAnalysisManager &) {
        LLVMContext &C = M.getContext();

        for (Function &F : M) {
            if (F.isDeclaration()) continue;

            Type *retTy = F.getReturnType();
            if (!retTy->isIntegerTy()) continue;

            IntegerType *IT = cast<IntegerType>(retTy);
            Constant *oneConst = ConstantInt::get(IT, 1);

            for (BasicBlock &BB : F) {
                for (auto it = BB.begin(); it != BB.end(); ) {
                    Instruction &I = *it++;
                    if (ReturnInst *RI = dyn_cast<ReturnInst>(&I)) {
                        IRBuilder<> B(RI);
                        B.CreateRet(oneConst);
                        RI->eraseFromParent();
                    }
                }
            }
        }
        return PreservedAnalyses::none();
    }
};
```

加载 BuggyReturnPass 编译

```
$ csmith > a.c
$ clang -O0 -fpass-plugin=./BuggyReturnPass.so a.c \
  -I /usr/include/csmith/ -o a
$ clang a.c -I /usr/include/csmith/ -o b
$ ./a > a.log
$ ./b > b.log
$ diff a.log b.log
1c1
< checksum = BFB85390
---
> checksum = 6B34DC6
```

探索空间

可考虑的扰动因素

- 多个编译器
- 同一个编译器的多个版本
- 同一个编译器的多个优化级别
- 等价模输入

CSmith 源码学习

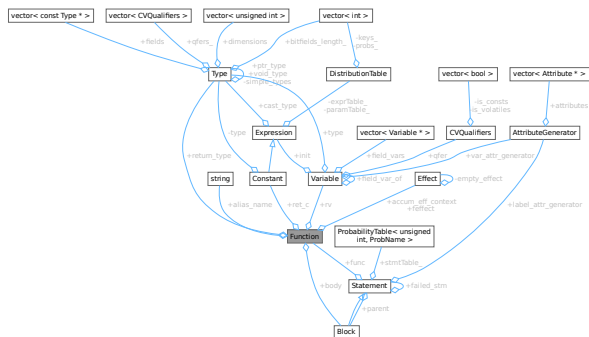


Figure 5: Function 类图

CSmith 源码学习

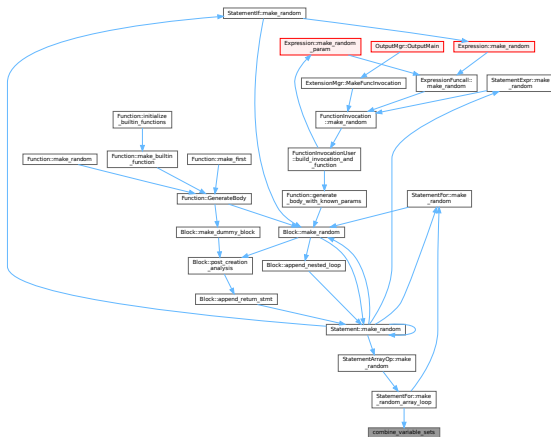


Figure 6: 函数构造调用图